

Introduction to Python



Goal: Quick reference for running Python, and using `print()` / `input()`.

Vocabulary

- **Python interpreter:** Program that executes Python code.
- **Interactive interpreter (REPL):** Runs one line at a time (great for quick tests).
- **Console / Terminal:** Text-based app to run commands and programs.
- **Command-line interface (CLI):** The command-based way you interact with the terminal.
- **Script file:** A file ending in `.py` that contains Python code.

How Python code runs

1. You write code in a **script file** (`.py`).
2. You run it using the **Python interpreter**.
3. The program can:
 - **Output** text using `print()`
 - **Read input** from the user using `input()`

Output: `print()`

Common patterns

```
print("Hello")
print(variable_name)
print(variable_name, "...", variable_name)
```

Notes

- Each `print()` starts output on a **new line** by default.
- Python lines **do not** end with a semicolon (`;`).
- Use `#` for **comments**.

New lines

- `\n` moves output to the next line.

```
print("Line 1\nLine 2")
```

Print on the same line (change `end`)

```
print("Hello there.", end=" ")
print("My name is Armando.")
# Output: Hello there. My name is Armando.
```

Input: `input()`

Basic input

```
best_friend = input()
```

- `input()` reads text from the user and returns a **string**.

Prompting the user

```
name = input("Enter your name: ")
```

Converting input to numbers

```
hourly_wage = int(input("Enter hourly wage: "))
```

- Wrapping `input(...)` with `int(...)` converts the string to an integer.
- Example of converting an existing string to an int:

```
number = int("15")
```



Remember: `input()` → string by default. Use `int(...)` (or `float(...)`) when you need a number.